server-side. This is even true when the resource is protected indirectly, as in this case, where you're accessing a Task that belongs to the Project, which is where the access control is attached. Second, you need to discuss how to test such a case.

You have two similar cases to deal with—task creation from the project form via TaskController#create and any of the update and move task methods in the controller.

Let's plan the create test. For the *given* you need a user, a project that the user belongs to, and a project the user doesn't belong to. The *when* is the creation of the task; the *then* is whether the task is created.

The design question is where to put the access check, and by extension where to write the test. You're at a slight disadvantage; since the potentially malicious request is coming from outside the UI, Capybara isn't going to be effective in crafting an integration test. You also didn't really write the full add-task feature before; you just kind of assumed its existence.

In other words, you can't easily write an integration test for this because the situation is outside the regular UI—if users don't have the ability to create a task, they also don't have the ability to see the project page. So, I somewhat reluctantly turn to a request spec. If the functionality were much more complex, I'd probably add Pundit and unit-test the Pundit object:

security/05/spec/requests/task_requests.rb

```
require "rails_helper"

RSpec.describe "task controller requests" do
  let(:project) { create(:project, name: "Project Bluebook") }
  let(:user) { create(:user) }

  describe "creation" do
    before(:example) do
      login_as(user)
    end

    it "can add a task to a project the user can see", :js do
      Role.create(user: user, project: project)
      post(tasks_path, params: {task: {name: "New Task", size: "3"}})
      expect(request).to redirect_to(project_path(project))
    end

    it "can not add a task to a project the user can see", :js do
      post(tasks_path, params: {task: {name: "New Task", size: "3"}})
      expect(request).to redirect_to(new_user_session_path)
    end

  end
end
```